

Title: Supporting `offsetof` for All Classes
Document number: P0897R0
Date: 2018-01-05
Project: ISO JTC1/SC22/WG21: Programming Language C++
Audience: EWG, LEWG
Reply-to: Andrey Semashev <andrey.semashev at gmail dot com>

1. Introduction

This proposal extends the set of types `offsetof` is required to support to all classes. It builds upon the feedback received for the [P0545R0](#) (Supporting `offsetof` for Stable-layout Classes) proposal. Its presentation in Albuquerque in 2017-11 resulted in general acknowledgement that the described problems seemed worth solving, and the subsequent discussion provided guidance toward a more comprehensive solution. The suggested avenues for further development were to broaden the set of types supported by `offsetof` either by relaxing standard-layout class restrictions or by specifying `offsetof` for more categories of types, without introducing new ones. While relaxing standard-layout class may be useful on its own, it does not coincide with the intention of the original proposal, which was to improve `offsetof`. Also, changing specification of standard-layout class might require coordination of more parts of the standard and possibly have effect on compatibility with C. Therefore this proposal takes the second approach. It does not preclude future proposals regarding standard-layout classes.

This paper inherits much of the motivation presented in [P0545R0](#) and refers to that document to avoid duplication. Some of the sections from that proposal are updated by this proposal, as noted in the text.

2. Impact on the Standard

This proposal is a pure extension to the C++ language and standard library. Its intent is to define behavior that was previously not defined, in a way that most current implementations already work. No existing valid portable code is made invalid or changes its behavior.

3. Design Decisions

3.1. Trivially-copyable and Non-trivially-copyable Types

[P0545R0](#), Sections 4.1 and 4.2, described that trivially-copyable types are already de-facto required to be compatible with `offsetof` by the current standard. Presence of non-trivial copy constructors and assignment operators also pose no difference to the object data layout in the current implementations. This makes those categories of types an easy extension.

3.2. Classes with Virtual Functions and/or Virtual Base Classes

Unlike [P0545R0](#), this proposal aims to add `offsetof` support for all classes, including those with virtual functions and virtual base classes. Although such classes are not suitable for data exchange between processes, there may be use cases where `offsetof` would still be useful. For example, an algorithm supporting heterogenous input (e.g. elements of different types in a heterogenous intrusive sequence) could be implemented more efficiently if it had information about offsets to the relevant data in the elements of the sequence. The

alternative implementations typically involve some sort of visitation and dynamic dispatch, which is likely more expensive in terms of performance and code size than pointer manipulation using offsets.

The most common implementation of virtual functions involves a virtual function table (vftable), a pointer to which is stored as a hidden data member in class objects. Like virtual functions, virtual base classes also typically introduce a base offset table (vbtable) and a hidden data member that points to it. This member is used to obtain the address of the virtual base class subobject at runtime. When both virtual functions and virtual base classes are used, some compilers may add yet another hidden member (vtordisp). The size and position of these hidden members are constant, so they do not preclude `offsetof` from working as expected.

It is possible that other implementations of virtual functions or virtual base classes exist. However, the author is not aware of implementations that make non-static data member layout for a given class a runtime property that is not available at compile time. Such a design would complicate ABI definition and likely have significant performance consequences with no apparent benefits. Therefore, the author considers such a design unlikely to be implemented and not worth allowing for, compared to the benefit from the improved support for `offsetof`.

Provided the above, it is possible to calculate the offset of a data member at compile time given that the type of the most derived object is known. In this case the compiler has full knowledge of layout of non-static data members in the object and does not require runtime resolution of the address of the base class subobject. Fortunately, the definition of `offsetof` given in the [C standard](#) (7.19/3), to which C++ refers in [support.types.layout]/1, is compatible with this precondition:

The macros are [...]; and

```
offsetof(type, member-designator)
```

which expands to an integer constant expression that has type `size_t`, the value of which is the offset in bytes, to the structure member (designated by *member-designator*), from the beginning of its structure (designated by *type*). The type and member designator shall be such that given

```
static type t;
```

then the expression `&(t.member-designator)` evaluates to an address constant. (If the specified member is a bit-field, the behavior is undefined.)

There are no base subobjects in C, so in terms of C++ the structure *type* can be only the most derived class. Also, the object `t` in this definition is the complete object of type *type*.

3.3. Abstract Classes

Complete objects of abstract classes cannot be created, which puts them in conflict with the literal `offsetof` definition given in the C standard and quoted above. However, the object that is used in the definition is only given for the purpose of the definition itself; invoking `offsetof` does not result in creation of the object of class *type*. The presence of pure virtual functions does not affect data layout compared to regular virtual functions.

The result of `offsetof` applied to an abstract class can be useful, if it can be used to adjust a pointer to a base subobject of that class. For example:

```
struct Base
{
    int n;

    virtual void foo() = 0;
};

struct Derived1 : public Base
```

```

{
    float x;

    void foo() override;
};

struct Derived2 : public Derived1
{
    std::string str;

    void foo() override;
};

Derived1 d1;
Derived2 d2;

constexpr std::size_t offset = offsetof(Base, n); // ok, works as if Base::foo was not marked
int* p1 = reinterpret_cast<int*>(reinterpret_cast<std::byte*>(static_cast<Base*>(&d1)) + offs
int* p2 = reinterpret_cast<int*>(reinterpret_cast<std::byte*>(static_cast<Base*>(&d2)) + offs

```

This becomes possible if we require base class subobjects to be contiguous and have the same layout in every most derived object (section 3.4 expands more on that). This requirement is currently fulfilled in every implementation the author is familiar with, unless the base class has virtual base classes itself. In this case the placement of the virtual base subobject in the storage typically depends on the most derived object size and layout. But when the most derived object has no non-static data members or other base classes of non-zero size (i.e. its size is equal to the base class subobject), the `offsetof` result is still valid.

```

struct BaseData
{
    int n;
};

struct Base : virtual public BaseData
{
    virtual void foo() = 0;
};

struct Good : public Base
{
    // No non-static data members or base classes other than Base, sizeof(Good) == sizeof(Base)

    void foo() override;
};

struct Bad : public Base
{
    float x;

    void foo() override;
};

Good g;
Bad b;

constexpr std::size_t offset = offsetof(Base, n); // still ok, works as if Base::foo was not marked
int* p1 = reinterpret_cast<int*>(reinterpret_cast<std::byte*>(static_cast<Base*>(&g)) + offset)
int* p2 = reinterpret_cast<int*>(reinterpret_cast<std::byte*>(static_cast<Base*>(&b)) + offset)

```

The author believes that `offsetof` is still useful with classes like `Good`. The broken uses with classes like `Bad` could potentially be diagnosed with a warning, if the compiler is able to track the actual type of the pointed object. However, the warning is not a requirement imposed by this proposal and is left as an aspect of quality of implementation.

3.4. Object Storage and Data Member Layout

In order for `offsetof` to be able to operate and provide a useful result, certain requirements must be fulfilled:

- objects of a class compatible with `offsetof` must occupy a contiguous region of storage, and
- offsets to non-static data members of such class must be constant and not depend on a particular most derived object of that class (this proposal refers to this property as *offset stability* or *stability of offsets*).

The first requirement allows to obtain the offset information and then apply it to an object and the second requirement guarantees that the information can be applied to any most derived object of that class.

Currently, the standard guarantees contiguous storage only for objects of trivially copyable or standard-layout types ([intro.object]/7). This proposal requires contiguous storage for objects of any type, except for objects that have virtual base class subobjects. Virtual base class subobjects may occupy storage bytes that are not immediately adjacent to the containing object, if that object is also a base class subobject. For example:

```
struct X
{
    int a;
};

struct V : virtual public X
{
    int b;
};

struct Y : public V
{
    int c;
};

V v;
Y y;
```

Both `v` and `y` are required to occupy contiguous bytes of storage, but the subobject of `y` that represents the base class `v` may not be stored contiguously if the implementation chooses the following data layout:

```
-----
|*V*|   includes V::b
-----
| Y |   includes Y::c
-----
|*X*|   includes X::a
-----
```

In this layout, the storage used by the base class `v` subobject is marked with asterisks (*).

Note also that the requirement of contiguous storage does not mean that any implementation-specific data such as virtual function tables or virtual base displacement tables need to be placed in the object's storage. These kinds of data do not contribute to the object's value representation ([basic.types]/4) and therefore don't need to occupy the object storage.

The standard has no explicit provision about stability of offsets of non-static data members across different objects of a class. For trivially copyable classes the data member offsets are required to be stable, as discussed in [P0545R0](#). For standard-layout classes the current definition of `offsetof` implies that members of such classes must have stable offsets. By extending `offsetof` to support other class types, this proposal requires that member offsets are stable for all classes.

3.5. Reference Data Members

The definition of `offsetof(type, member-designator)` in the C standard only requires `&(t.member-designator)` to evaluate to an address constant (here, `t` is an object of class type). Since this proposal allows using `offsetof` with non-trivial types, `member-designator` can now identify a reference member. Taking the address of a reference member would result in the address of the referred object, which is not the intended effect of `offsetof`. For this reason, this proposal prohibits the use of references in a `member-designator`; the behavior is undefined if this requirement is violated. Note that this rule applies if references are used at any level of `member-designator`, if it identifies a nested member. For example:

```
struct A
{
    int a;
};

struct Bad
{
    int& n;
    A& a;
    int x;
};

offsetof(Bad, n);    // undefined behavior, Bad::n is a reference
offsetof(Bad, a.a); // undefined behavior, Bad::a is a reference
offsetof(Bad, x);    // ok, returns offset of Bad::x
```

4. Impact on Existing Implementations

The currently widespread compilers (GCC 7.2, Clang 4.0.1, MSVC 19.12.25831) all support `offsetof` for trivially copyable and non-trivially classes, as well as for classes with virtual functions (including pure virtual functions), although some of the compilers emit warnings, as currently required by [N4713](#). This proposal makes all classes supported unconditionally, which will remove the need for a diagnostic.

The tested compilers also have limited support for `offsetof` with classes with virtual base classes.

```
struct A
{
    int a;
};

struct D : virtual public A
{
    int c;
    char str[1024];
};

offsetof(D, a);    // (1)
offsetof(D, c);    // (2)
```

```
offsetof(D, str); // (3)
offsetof(D, str[10]); // (4)
```

In the above example, all three tested compilers were able to compile all `offsetof` expressions, except (1), which is the only one that requests an offset of a member of a virtual base class. The updated compilers would have to be able to calculate the offset to members of virtual base classes at compile time.

Other than case (1), all tested compilers appeared to return values that are aligned with this proposal (i.e. the returned values were equal to the offset to the requested member in a *complete* object of the specified type). Therefore, even programs that previously relied on implementation-specific behavior of `offsetof` are unlikely to be affected.

5. Proposed Wording

The proposed wording below is given relative to [N4713](#). Inserted text is marked like this, removed text is marked like ~~this~~.

5.1. Core Wording

Modify [intro.object]/7:

Unless it is a bit-field (12.2.4), a most derived object shall have a nonzero size and shall occupy one or more bytes of storage. Base class subobjects may have zero size. ~~A most derived object of trivially-copyable-or-standard-layout type (6.7)~~ or a base class subobject of nonzero size excluding any of its virtual base class (13.1) subobjects shall occupy contiguous bytes of storage. A virtual base class subobject may occupy a distinct non-adjacent contiguous region of storage from the rest of its containing base class subobject. Each such region of storage shall be within the enclosing contiguous region of storage occupied by the containing most derived object.

5.2. Library Wording

Modify [support.types.layout]/1:

The macro `offsetof(type, member-designator)` has the same semantics as the corresponding macro in the C standard library header `<stddef.h>`, ~~but accepts a restricted set of type arguments in this International Standard. Use of the `offsetof` macro with a type other than a standard layout class (Clause 12) is conditionally supported~~ with the following additions. The argument `type` is interpreted as the most derived class (6.6.2). [Note: For the purpose of this definition, all pure-specifiers (12.2) in member function declarations of `type` and its base classes are ignored. — end note]. The expression `offsetof(type, member-designator)` is never type-dependent (17.7.2.2) and it is value-dependent (17.7.2.3) if and only if `type` is dependent. The result of applying the `offsetof` macro to a static data member or a function member is undefined. Given an object `t` of type `type`, if `t.member-designator` accesses or identifies a reference data member, the result of the `offsetof` expression is undefined. No operation invoked by the `offsetof` macro shall throw an exception and `noexcept(offsetof(type, member-designator))` shall be true.

6. Acknowledgements

- Thanks to Walter E. Brown for the help with preparing and presenting the proposal.

7. References

- P0545R0 proposal, Supporting offsetof for Stable-layout Classes (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0545r0.html>)
- N4713 Working Draft, Standard for Programming Language C++ (<http://open-std.org/JTC1/SC22/WG21/docs/papers/2017/n4713.pdf>)
- N1570 Working Draft, Programming languages - C (<http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf>)